

Character Movement & Collision Parity Audit

Godot Explorer vs. Unity Explorer

Research deliverable for issue #905 · docs-only, no behavior change

August 25, 2026

Abstract

Players cannot climb or jump onto geometry that is trivially reachable in the Unity desktop client — the same scene plays as a different game. This document is the full audit requested by issue #905: every character-movement and collision parameter of godot-explorer was diffed against unity-explorer (the shipping reference), each gap was verified against source code on both sides, classified as *parity bug*, *missing system*, or *unconfirmed deviation*, and turned into a prioritized follow-up roadmap. Sources: godot-explorer `main` @ `7f7302805c`; unity-explorer `dev` @ `6d8b5e966`.

Contents

1	Scope and methodology	2
1.1	What was audited	2
1.2	Sources	2
1.3	Verification method	2
2	The avatar capsule (cross-reference only)	2
3	Gap analysis by area	3
3.1	Step, ground contact, and tick model	3
3.2	Jump and gravity	3
3.3	Glide	3
3.4	Speed, acceleration, and air behavior	4
3.5	Slope, edge, wall, landing, platforms	5
3.6	Collider generation and layers	5
4	Classification	6
4.1	Parity bugs — Godot behavior is wrong	6
4.2	Missing systems — exist in Unity, must be built	6
4.3	Unconfirmed deviations — need a product call	6
4.4	Warning: Unity dead configuration	6
5	Reproduction checklist (test scene -98,103)	6
6	Salvage decisions (recommendations)	7
7	Jolt evaluation	7

8 Follow-up roadmap (not yet filed) 7

A Verification log 7

1 Scope and methodology

1.1 What was audited

All character-movement and collision surfaces: avatar capsule dimensions and offset, step offset / floor snap / slope limit, coyote time and jump buffering, jump heights and the gravity model, speeds / acceleration / air control / air drag, slope, edge and wall behavior, glide, GLTF and primitive collider generation, collision layers and masks, and the physics engine and tick model.

1.2 Sources

- **Godot side:** `godot/src/logic/player/player.gd` + `player.tscn` (all locomotion), `lib/src/content/gltf/scene.rs` and `lib/src/scene_runner/components/mesh_collider.rs` (collider generation), `godot/project.godot` (layer names), `graphic_settings.gd` (tick rate).
- **Unity side:** `CharacterControllerSettings.asset` (authoritative tuning), `CharacterObject.prefab` (the capsule), the `CharacterMotion` systems (`ApplyJump`, `ApplyGravity`, `ApplyGliding`, `ApplyEdgeSlip`, `ApplyWallSlide`, ...), `PhysicsLayers.cs` and `DynamicsManager.asset` (layer routing and collision matrix).

1.3 Verification method

Every quoted value carries a file:line reference and was verified by reading the source at that location, in two independent adversarial review passes instructed to disprove each claim (see Appendix).

2 The avatar capsule (cross-reference only)

Capsule parity is tracked in its own issue; it is summarized here because several other gaps are side effects of it.

	Godot	Unity
Height / radius	1.5 / 0.25	1.6 / 0.3
Center Y	1.2	0.8
Margin / skin width	0.05	0.08
Extra shape	<code>SeparationRayShape3D</code> length 1.0779 at $y=1.1$	none

The Godot capsule floats 0.45 m above the feet and a downward separation ray bridges the gap. This one difference masquerades as several others: Godot’s “de-facto step clearance” of ≈ 0.45 m is not step logic, it is the floating capsule rolling over low geometry.

3 Gap analysis by area

3.1 Step, ground contact, and tick model

	Godot	Unity
Step offset	none — no built-in, no custom logic	0.35, force-written every physics tick (<code>ApplySlopeModifier.cs:24</code>); prefab serializes 0.3
Floor snap / downslope stick	<code>floor_snap_length = 0.2</code> (<code>player.gd:218</code>)	downslope raycast 0.45 jog / 0.55 run (<code>ApplySlopeModifier.cs:40,42</code>)
Slope limit	unset → engine default 45°	46° (<code>CharacterObject.prefab:103</code>)
Physics engine / tick	Godot Physics, 60 Hz (<code>graphic_settings.gd:235,243</code>)	PhysX, manual variable timestep <code>Physics.Simulate(dt)</code> per frame (<code>UpdatePhysicsSimulationSystem.cs:33</code>)

Why it matters. The missing step offset is the single largest “can’t climb trivial geometry” contributor once the capsule is fixed: Unity walks up anything up to 0.35 m; Godot must jump. The tick finding is newer and easy to miss: Unity no longer runs a fixed 50 Hz simulation — it steps physics by the frame delta. Any reasoning of the form “coyote time = 8 ticks” is nominal, not guaranteed, and per-tick logic on the Godot side (which *is* fixed at 60 Hz) will drift relative to Unity under load.

3.2 Jump and gravity

	Godot	Unity
Coyote time	unreachable: grace (<code>player.gd:391-395</code>) only drives the land animation; ground-jump branch (<code>:455-470</code>) sits after <code>elif</code> <code>not on_floor (:390)</code>	0.15 s $\approx$ 8 nominal ticks (<code>ApplyJump.cs:54</code>)
Jump buffer	0.15 s (<code>player.gd:9</code>)	0.15 s (<code>JumpTrigger.cs:8-9</code>) ✓
Gravity	single 10 m/s ²	asymmetric: ascent $\times 4 = 40$, descent 10 (<code>ApplyGravity.cs:41-42</code>)
Long jump (hold)	none	0.5 s window, $\times 0.5$ gravity (<code>ApplyGravity.cs:37-38</code>)
Jump height	1.8 fixed (<code>player.gd:59-60</code>)	1.0 jog / 1.5 run, lerped by horizontal speed
Air jump	1 · 2.0 m · 0.2 s delay · fixed impulse 8.0 (<code>player.gd:8,11-13</code>)	same values, impulse max(8, current speed) (<code>ApplyJump.cs:76-186</code>); behind <code>FeatureId.DoubleJump</code>
Jump cooldown	0.3 s (<code>player.gd:10</code>)	0.3 s (<code>ApplyJump.cs:80-83</code>) ✓

Why it matters. The unreachable coyote time (**B1**) is the worst feel bug: a jump pressed within 0.15 s of walking off a ledge — muscle memory for every platformer player — is silently dropped in Godot, and off higher ledges the same input is instead consumed by the glide-open gate, compounding the confusion. The asymmetric gravity plus long-jump hold is what makes Unity jumps feel snappy going up and forgiving coming down; Godot’s symmetric single gravity cannot reproduce that arc no matter how the height constant is tuned. Air jump and jump cooldown match exactly — both systems exist on dev with identical parameters.

3.3 Glide

Both engines have a glide system.

	Godot	Unity
Max fall speed	1.0 (<code>player.gd:14</code>)	1 (<code>ApplyGliding.cs:55</code>) ✓
Horizontal speed	cap 6.0 (<code>player.gd:15</code>)	<code>GlideSpeed</code> 6 (<code>CalculateSpeedLimitSystem.cs:36</code>) ✓
Min ground distance	1.0 (<code>player.gd:16</code>)	0.2 (<code>settings :84</code>)
Jump→glide interval	0.5 (<code>player.gd:17</code>)	0.5 (<code>settings :87</code>) ✓
Glide cooldown	0.6 (<code>player.gd:18</code>)	0.2 (<code>settings :88</code>)
Trigger model	toggle-press; walking off a ledge can open glide (deliberate divergence)	hold-to-glide after air jumps exhausted
Wind response	none	<code>GlideWindResponse</code> 1.5 (<code>ApplyExternalForce.cs:29</code>)

Why it matters. Two plain parameter mismatches (**B5**, **B6**): Godot refuses to open the glider below 1 m of altitude where Unity allows 0.2 m, and enforces a $3\times$ longer re-open cooldown. Both are two-constant fixes with immediate playability impact around rooftops and ledges.

3.4 Speed, acceleration, and air behavior

	Godot	Unity
Walk / jog / run	1.5 / 8 / 11	1.5 / 8 / 10
Acceleration	none — velocity assigned directly (<code>player.gd:513-514</code>)	ground 20→25, air 15→20, over a 0.5 s curve (<code>ApplyCharacterMovementVelocity.cs:72-78</code>)
Deceleration	<code>move_toward(v, 0, walk_speed)</code> per tick, not dt-scaled $\Rightarrow$ 90 m/s ² at 60 Hz (<code>player.gd:520-521</code>)	<code>SmoothDamp</code> with <code>StopTimeSec = 0</code> (effectively instant)
Air drag	none on locomotion velocity	quadratic, coeff $0.05 \times 4 = 0.2$ (<code>ApplyHorizontalAirDrag.cs</code>)
Air control	full, identical to grounded	reduced: air accel pair + <code>MoveTowards</code>

Why it matters. Godot’s instant velocity assignment and full air control make the character feel “on rails” compared to Unity’s weight. The deceleration line hides a latent bug (**B4**): the per-tick step is not scaled by `delta`, so stopping distance depends on the physics tick rate — fine at a stable 60 Hz, wrong under load.

3.5 Slope, edge, wall, landing, platforms

System	Godot	Unity
Slope velocity modifier	none	1.35× downhill → 0.65× uphill (<code>ApplyCharacterMovementVelocity.cs:21</code>)
Steep-slope slide	none	gravity tilted along slope beyond 46° (<code>ApplyGravity.cs:20-29</code>)
Edge slip	none	spherecast + gravity-tilt slip (<code>ApplyEdgeSlip.cs:28-31,71</code>)
Wall slide	none	capsulecast 0.5 m, multiplier lerp'd to 0 (<code>ApplyWallSlide.cs:29-42</code>)
Hard-landing stun	present: scene-driven cooldown (<code>player.gd:304-308</code>)	fall > 8 m ⇒ 0.75 s stun (<code>StunCharacterSystem.cs:44-80</code>)
Moving platforms	partial: colliders go KINEMATIC, no follow logic	platform delta + rotation applied (<code>CharacterPlatformSystem.cs</code> , <code>PlatformSaveLocalPosition.cs:28-34</code>)

Corrections to the #905 baseline. Hard-landing stun is *not* absent in Godot — what is missing is Unity’s fall-height trigger. Moving platforms are *partial*, not absent. Edge slip in Unity works by tilting the gravity direction, *not* by the `EdgeSlipSpeed` constant, which is dead configuration.

3.6 Collider generation and layers

	Godot	Unity
GLTF colliders	trimesh per mesh (<code>scene.rs:169</code>)	<code>MeshCollider</code> , <code>BakeMesh</code> (<code>ConfigureGltfContainerColliders.cs:93-100</code>) ✓
Backfaces	only non-planar meshes; planar = AABB thinner than 0.01 (<code>scene.rs:219,227-229</code>)	always double-sided
Primitive colliders	box/sphere/cylinder/plane shapes	same mapping ✓
Player mask	CL_PHYSICS only (<code>player.tscn:20</code>)	6 layers incl. <code>InvisibleColliders</code> , <code>CharacterOnly</code> , <code>SDKAvatarTriggerArea</code>
SDK layer routing	6 functional + 13 reserved/custom layers	incl. avatar-only → <code>SDKAvatarHit</code> route (<code>PhysicsLayers.cs:123-127</code>)

Why it matters. Two of the six parity bugs live here. **B2:** Godot marks any mesh thinner than 1 cm on any axis as one-way, so thin floors and ramps collide only from above — players fall through geometry that is solid in Unity (direct cause candidate for #1529 and #1203). **B3:** the Godot player collides with a single layer, so scene-placed invisible blockers and several SDK collider categories simply do not exist for the character.

4 Classification

4.1 Parity bugs — Godot behavior is wrong

#	Gap	Evidence
B1	Coyote time unreachable (elif-chain; input eaten by glide gate)	<code>player.gd:390</code> vs <code>:455-470</code>
B2	One-way thin colliders	<code>scene.rs:219,227-229</code>
B3	Player mask 1 layer vs 6	<code>player.tscn:20</code>
B4	Deceleration not dt-scaled	<code>player.gd:520-521</code>
B5	Glide min ground distance 1.0 vs 0.2	<code>player.gd:16</code>
B6	Glide cooldown 0.6 vs 0.2	<code>player.gd:18</code>

4.2 Missing systems — exist in Unity, must be built

M1 step offset · M2 slope velocity modifier · M3 steep-slope slide · M4 edge slip · M5 wall slide · M6 platform follow · M7 acceleration curves + air control · M8 air drag · M9 long jump · M10 asymmetric gravity · M11 fall-height stun · M12 glide wind response · M13 avatar-only collider layer.

4.3 Unconfirmed deviations — need a product call

D1 run 11 vs 10 · D2 jump 1.8 fixed vs 1.0/1.5 speed-lerped · D3 snap 0.2 vs raycast 0.45/0.55 · D4 slope 45° vs 46° · D5 stun trigger model · D6 air-jump impulse fixed 8.0 vs $\max(8, \text{speed})$ · D7 glide trigger model (partly deliberate — there is a divergence comment in `player.gd`).

4.4 Warning: Unity dead configuration

Replicate *behavior*, not asset values. Dead or misleading on the Unity side: `MinSlopeAngle 50` (real steep threshold is the 46° slope limit), `EdgeSlipSpeed 2` (slip is gravity-tilt), `StopTimeSec=0` (degenerate `SmoothDamp` = instant stop), `GroundDrag`, `MinImpulse`, `CharacterControllerRadius 0.4`, and the entire `ApplyAirDrag.cs` file (the live one is `ApplyHorizontalAirDrag.cs`). The prefab serializes `StepOffset 0.3` but code overwrites it with 0.35 every tick. `MaxSlopeAngle 80` is consumed (wall slide, slide animation, conditional rotation) — just not for the steep-slope trigger.

5 Reproduction checklist (test scene -98,103)

Derived from code, not yet executed live. Side-by-side, Unity vs Godot:

- **B1** — walk off a low ledge, jump within 0.15s: Unity jumps; Godot drops the input.
- **B2** — approach thin elevated floors from below: Unity blocks; Godot passes through.
- **B3** — walk into invisible blocker walls: Unity blocks; Godot walks through.
- **B4** — release keys at full run under frame drops: Unity stops consistently; Godot varies with tick rate.
- **B5/B6** — open glider below 1 m altitude, then close and re-open quickly: Unity opens at 0.2 m and re-arms in 0.2s; Godot refuses and waits 0.6s.
- **M1** — walk into 0.35–0.5 m ledges: Unity steps up; Godot blocks.
- **M2/M3** — run up vs down a long ramp; stand on a >46° slope: Unity modulates speed and slides; Godot does neither.

- **M9/M10** — tap vs hold jump: Unity arc differs; Godot arc is fixed and symmetric.
- **D1/D2** — run-jump onto Unity-reachable geometry: different reachable set (1.5 m @ 10 m/s vs 1.8 m @ 11 m/s).

6 Salvage decisions (recommendations)

PR #943 — Rust locomotion rewrite + Jolt: drop the code, keep the body. Locomotion lives in `player.gd` and is iterated there; a Rust rewrite is the wrong shape for incremental parity. The PR body, however, is an ADR-grade locomotion spec — reuse it as the implementation checklist for M1–M13.

PR #1417 — capsule + snap + slope + coyote + Jolt cherry-pick: re-scope without Jolt and reopen. The minimal parity slice. Split the engine change out, land the GDScript-side fixes (coyote B1, snap/slope values) and the capsule fix via its own issue.

7 Jolt evaluation

Both dead PRs bundled the Jolt physics engine. Recommendation: **do not bundle Jolt with parity fixes**. All six parity bugs and most missing systems are solver-independent script logic. Jolt would improve contact robustness around thin geometry, but it does not fix B2 (the one-way flag is our code, not the engine), and it imposes a cross-platform revalidation cost. Bundling an engine swap with behavior changes makes regressions unattributable — the failure mode that killed both prior PRs. Re-evaluate Jolt only if thin-geometry tunneling survives the B2 fix.

8 Follow-up roadmap (not yet filed)

Pri	Candidate	Addresses	Effort
P0	Fix coyote elif-chain	B1	XS
P0	Double-sided thin colliders	B2, #1529, #1203	S
P0	Widen player collision mask	B3	XS
P1	dt-scale deceleration	B4	XS
P1	Glide param alignment (0.2 / 0.2)	B5, B6	XS
P1	Asymmetric gravity + long jump + lerp jump height	M9, M10, D2	S
P1	Step offset (custom)	M1	M
P2	Slope modifier + downslope raycast	M2, D3	S
P2	Steep slide + edge slip + wall slide	M3–M5	M
P2	Acceleration curves + air control + air drag	M7, M8	M
P3	Platform follow	M6	M
P3	Fall-height stun	M11, D5	S
P3	Glide wind + avatar-only collider layer	M12, M13	S
—	Jolt spike (only after B2 lands)	§7	spike
—	Product decision on deviations	D1–D7	—

A Verification log

Two independent adversarial validation passes were run against this document (2026-08-25), each instructed to disprove every claim:

- An initial pass ran against a stale local unity-explorer checkout and produced false positives (“Unity has no air jump”, “no jump cooldown”) — both systems exist on `dev` with values matching Godot exactly. All Unity claims were re-validated against the updated checkout (`dev @ 6d8b5e966`).
- Validation also surfaced: the glide system on both sides with two real parameter mismatches (B5/B6); `MaxSlopeAngle` consumed (not dead) vs. `EdgeSlipSpeed` dead; Unity physics running a manual variable timestep rather than fixed 50Hz; the layer-16 matrix contents (`SDKAvatarTriggerArea` in); and the dead `ApplyAirDrag.cs` file (live: `ApplyHorizontalAirDrag.cs`).
- Baseline claims from issue #905: 13/13 Godot values verified exact. Corrections: hard-landing stun exists (scene-driven); moving platforms are partial (KINEMATIC switch without follow).